

COMP101

Introduction to Programming

2025-26

Lecture-11

Modules & Functions

Python Library

Programming complicated code

- Sometimes we need something complicated in our code to solve a problem
- e.g. pi is frequently used across a large range of applications - it is important to calculate it correctly and appropriately
- We could program a solution to it ourselves and use our solution for pi in our code for the problem

pi – sample design to program it

- Inscribe a regular hexagon inside a circle
- Circumscribe a regular hexagon outside the same circle
- Calculate the circumference and diameter of the hexagons to give:

$$\pi = \text{circumference} / \text{diameter}$$

- Analysis
 - decimal places, accurate, best algorithm ...

Code re-use

- Re-using code is an important aspect of programming
- We don't have to calculate pi ourselves every time we want to use it
- We can code it, save it and then 'call' the programme whenever we want to use pi

Code re-use

- The saved program (a .py file) is a 'module'
- The calculation of pi in it is a 'function'
- Important functions (e.g. pi etc) have been done by experts and fully tested for accuracy
- A 'module' (a .py file) usually has several functions in it, e.g.
 - pi, random, radians etc

Functions

- Certain functions are in constant use in a program and can be used efficiently
- Python automatically loads a set of these 'builtin' functions due to their common use
- Functions are discreet Python programs
 - They are pre-programmed and tested by experts
 - We have used some already: `input()`, `print()` etc

Functions

- Function names have empty parentheses after them – this is where we put the data we want the function to work on
- Functions that load with Python are referred to as ‘builtins’ and are always available
- <https://docs.python.org/3/library/functions.html>
- Actual function list can vary by Python version and by installation

Functions that load with Python

<https://docs.python.org/3/library/functions.html>

<u>abs()</u>	<u>dict()</u>	<u>help()</u>	<u>min()</u>	<u>setattr()</u>
<u>all()</u>	<u>dir()</u>	<u>hex()</u>	<u>next()</u>	<u>slice()</u>
<u>any()</u>	<u>divmod()</u>	<u>id()</u>	<u>object()</u>	<u>sorted()</u>
<u>ascii()</u>	<u>enumerate()</u>	<u>input()</u>	<u>oct()</u>	<u>staticmethod()</u>
<u>bin()</u>	<u>eval()</u>	<u>int()</u>	<u>open()</u>	<u>str()</u>
<u>bool()</u>	<u>exec()</u>	<u>isinstance()</u>	<u>ord()</u>	<u>sum()</u>
<u>bytearray()</u>	<u>filter()</u>	<u>issubclass()</u>	<u>pow()</u>	<u>super()</u>
<u>bytes()</u>	<u>float()</u>	<u>iter()</u>	<u>print()</u>	<u>tuple()</u>
<u>callable()</u>	<u>format()</u>	<u>len()</u>	<u>property()</u>	<u>type()</u>
<u>chr()</u>	<u>frozenset()</u>	<u>list()</u>	<u>range()</u>	<u>vars()</u>
<u>classmethod()</u>	<u>getattr()</u>	<u>locals()</u>	<u>repr()</u>	<u>zip()</u>
<u>compile()</u>	<u>globals()</u>	<u>map()</u>	<u>reversed()</u>	<u>__import__()</u>
<u>complex()</u>	<u>hasattr()</u>	<u>max()</u>	<u>round()</u>	
<u>delattr()</u>	<u>hash()</u>	<u>memoryview()</u>	<u>set()</u>	

Module List

<https://docs.python.org/3/py-modindex.html>

- Sometimes we want a function that isn't in the 'builtin' function list
- These functions are stored in modules
- Modules are held in a 'Python Library'
 - They have a .py extension – basically they are Python programmes
 - They don't load automatically with Python
 - We have to import them ('call')

Modules

<https://docs.python.org/3/py-modindex.html>

- Import
 - The library holds many modules and more can be added from external sites. COMP101 only uses modules that come with Python Library
- Useful modules include
 - math
 - random
 - statistics
 - datetime



Remember, there are many modules and each module contains many functions:
We'll focus on the 'math' module

math module – function set

- 'acos', 'acosh', 'asin', 'asinh', 'atan', 'atan2', 'atanh', 'ceil', 'copysign', 'cos', 'cosh', 'degrees', 'e', 'erf', 'erfc', 'exp', 'expm1', 'fabs', 'factorial', 'floor', 'fmod', 'frexp', 'fsum', 'gamma', 'gcd', 'hypot', 'inf', 'isclose', 'isfinite', 'isinf', 'isnan', 'ldexp', 'lgamma', 'log', 'log10', 'log1p', 'log2', 'modf', 'nan', 'pi', 'pow', 'radians', 'remainder', 'sin', 'sinh', 'sqrt', 'tan', 'tanh', 'tau', 'trunc'

Import: Modules and Functions

- Put import statements at the start of the code

```
import math #imports math module and ALL its functions
```

```
#start program
```

```
print (math.pi)
```

```
3.141592653589793
```

We need to know the module name ('math') and the function name ('pi')

NOTES:

'dot notation' allows us use the function.

The module name comes first, then the dot, then the function name

Import: Modules and Functions

```
import math
```

```
num = int(input("Enter a number: ")) 16
```

```
#in module 'math' use function 'sqrt' using 'dot notation'
```

```
answer = math.sqrt(num)
```

```
print (answer)
```

```
4.0
```

We need to know that the name of the function that does square root is 'sqrt'

Import: Modules and Functions

```
import math
```

```
num = float(input("Enter a number: ")) 16.2
```

```
#round down to the nearest integer
```

```
answer = math.floor(num) #uses function 'floor' from module 'math'
```

```
print (answer)
```

```
16
```

```
#round up to nearest integer
```

```
next_answer = math.ceil(num)
```

```
print (next_answer)
```

```
17
```

Import: Modules and Functions

- We can import several modules

```
import math      #imports all functions in module math
```

```
import random    #imports all functions in module random
```

```
print (math.pi)  #dot notation selects only function 'pi'
```

```
dice_throw = random.randint(1,6) # needs two parameters to give a range  
print (dice_throw)
```

```
3.141592653589793 #pi
```

```
3                #randint generates a random number 1 to 6 inclusive
```

Import: Modules and Functions

- We can import specific functions

```
from math import pi
```

```
from random import randint
```

```
#no dot notation is needed
```

```
print (pi)
```

```
dice_throw = randint(1,6)
```

```
print (dice_throw)
```

```
3.141592653589793 #this is pi
```

```
5 #random number
```

We have not imported all the functions in modules 'math' and 'random' – only the pi() function in math module and the randint() function in random module

No dot notation needed for pi and randint as no other functions have been imported.

Can't use other functions, e.g. sqrt() etc as they are not imported here

Functions – builtin list

- builtin functions – see them all in the shell

```
>>>import (builtins)
```

```
>>> dir (builtins)
```

These are the functions that load with Python

They can be used ‘immediately’ in the code

i.e. we don’t need to import these

Module List

- pre-built modules – see them all in the shell

```
>>>help()
```

```
help> 'modules'    #note apostrophes
```

Must be imported before we can use them in Python code

Module – functions list by module

Must be imported before we can use them

- To get a function from a module we need to know:
 - name of function and name of module

e.g.

what functions are in the math module?

```
>>> import(math)
```

```
>>> dir (math) #shows all the functions available in the math module
```

Modules and Functions

May be easier/clearer to use the python.org site

e.g. see all functions in the math module etc

<https://docs.python.org › library › math>

Or try a search with 'Python math module' etc